

II1305 - Project Reflection: Product Owner Journey

Samia Serbouti

16th May 2026

Abstract

This reflection document synthesizes my experience as Product Owner of Team Riebnes during the development of the HouseBite application. The reflection draws directly from challenges encountered particularly around team dynamics, communication, and inclusive design.

1 Advice I would give to my past self

1.1 Establish technical proofs of concept early

Rather than allowing new design choices to appear during sprints, I would insist on creating a proof of concept or at least a technical spike for any uncertain decision within the first two weeks. For HouseBite, some architectural oversights became visible only midway through development, leading to reworks that could have been avoided. A PoC doesn't need to be production-ready, but it must answer the question: "Will this approach work given our constraints?"

1.2 Increase synchronization cadence

I underestimated how quickly assumptions diverge in a distributed team. I would try to think of daily check-ins (15 minutes each, no agenda except "what are you working on right now?") and maybe do a "walk the board" ritual at the start of every meeting (where each dev gives notes or suggestions on the sprint backlog). This simple habit would have caught several misalignments before they became issues.

1.3 Enforce a strong pull request culture from day one

The social contract is a perfect moment to define a precise code review protocol. Our team's eventual PR system worked nicely, but we implemented it only after two messy merges. I would now ask:

- Every PR requires two approvals before merging to `main`.
- Each PR must include a written explanation of changes and verify for lint errors and other compilation errors.

1.4 Take intentional breaks and seek external feedback

I would schedule two 15-minute breaks during any 4-hour coding block—and more importantly, I would force myself to walk to another team's table (or video call) once per week to quickly demo our latest feature and ask for raw, unfiltered feedback.

1.5 Address tensions immediately, not at retrospectives

This was my hardest lesson. Our team had two distinct cultures: members who valued strict punctuality and synchronous work, and others who preferred flexible schedules and “chill” pacing due to competing coursework. As weeks passed, small frustrations grew into tensions. As Product Owner, I tried my best to find compromises, but I waited too long to start those conversations. My advice would be to discuss a tension the moment you notice it using non violent communication skills i learned, and never rely solely on the sprint retrospective to heal relationships and communication.

2 Two good ways of working I would repeat

2.1 Tasks dependencies tracking with Excalidraw

I used Excalidraw, a simple virtual whiteboard, to visualize task dependencies before each sprint planning session. As a PO, it helped me tremendously with planning meetings, and rather than arguing about who should take which task, we could literally draw arrows from “User authentication” to “Profile page” to “Order history”. This visual map served several purposes:

1. Devs could refer to it instantly, it prevented them from coming to me asking to take a task that they clearly could not address right away which kept things easier for me.
2. It made blocking relationships undeniable—no one could claim their task was independent when the arrow said otherwise.
3. It guided task distribution: the developer blocked by the fewest dependencies started first.
4. During stand-ups, we could point to the diagram and say “I’m blocked here” with instant clarity for everyone.

I would absolutely use Excalidraw (or any collaborative diagramming tool) again. The only downside was that we used this system as well as a task board spreadsheet where we wrote what we were working on at the moment and a github backlog to view task progress status. Overall the three systems were a bit much, but we chose to keep them because each were useful to at least one person.

2.2 Disciplined pull request system

What made our system work was the *social contract enforcement*: we agreed that any merged PR without two approvals could not “corrupt” the deployed app. This created a sense of safety, devs with more experience were sure that things wouldn’t randomly break down, and the juniors did not have to stress so much about pushing their branches. It was also the perfect way to learn new things and see how our peers wrote code.

3 Two ways to improve this project

3.1 Mandatory user testing with external participants

While we conducted internal test sessions, every dev already understood the project’s assumptions. Real problems only show up when someone who has never seen the app tries to do something. A future iteration of the course should require:

- At least one round of user testing with participants who are *not* students in the course and have not been briefed (ideally, one test session held before the halfway point so fixes can actually be implemented).

- A short report documenting usability issues discovered and how they were fixed.

This would also train students in a core product owner skill: separating your own mental model from the user's reality.

3.2 Make the project explicitly inclusive

Our team rapidly discussed accessibility but never systematically implemented it. I would redesign the project rubric to include specific accessibility features.

4 Working remotely vs. on campus

4.1 Advantages of remote work

- Paradoxically, we talked more often remotely because messaging (on Zoom) was always open. A quick “hey, how does this component work?” or “can I show you how this page look like?”
- Voice channels enabled spontaneous subgroup discussions without disturbing the whole team.
- Our on-campus rooms hosted four groups in one room, creating constant background noise. At home, I could focus deeply.
- No one was watching me step away from my desk. I worked when I was sharp and rested when I was tired, which ultimately increased my productivity.
- Code reviews and debugging sessions became *better* than in-person because everyone saw exactly the same screen, and we could annotate without moving around a laptop.

4.2 Disadvantages of Remote Work

- Some students may lack stable internet, a quiet room, or an proper setup. Our team did not have this issue, but the course should not assume all students have equal remote-working conditions.
- Facial expressions and body language are muted (except when we were taking proper meetings so once or twice a day). A teammate quietly disengaging is easier to miss on video.

4.3 Recommendation for future iterations

I would love to see a hybrid model tested: two days on campus (for example sprint planning and sprint review) and two days remote (development).